

Views

-- Approach 1: using subqueries

```
SELECT p.ProductName, p.ModelName  
FROM products p  
WHERE (  
    SELECT SUM(r.ReturnQuantity)  
    FROM Returns r  
    WHERE r.ProductKey = p.ProductKey  
) > 50;
```

for each row

1,00,000 record

-- Approach 2: using joins

```
SELECT P.ProductName, P.ModelName  
FROM Products P  
JOIN  
(SELECT ProductKey, COUNT(*) AS ReturnQuantity  
FROM Returns  
GROUP BY ProductKey  
HAVING ReturnQuantity > 50) R  
ON R.ProductKey = P.ProductKey;
```

combine two tables together

```
SELECT P.ProductName, P.ModelName  
FROM Products P  
JOIN  
(SELECT ProductKey, COUNT(*) AS ReturnQuantity  
FROM Returns  
GROUP BY ProductKey) R  
ON R.ProductKey = P.ProductKey  
WHERE R.ReturnQuantity > 50;
```

More
optimized as

comparable to

the above one

In SQL, a view is a virtual table based on the result set of an SQL SELECT query. It does not store data physically but displays data stored in its underlying (base) tables dynamically when it is queried. Views are primarily used to simplify complex queries, enhance security, and present a customized perception of the database to users

Index → to access the elements
 from any data structure
 (List, Array)

B-tree, B+ tree

Customers

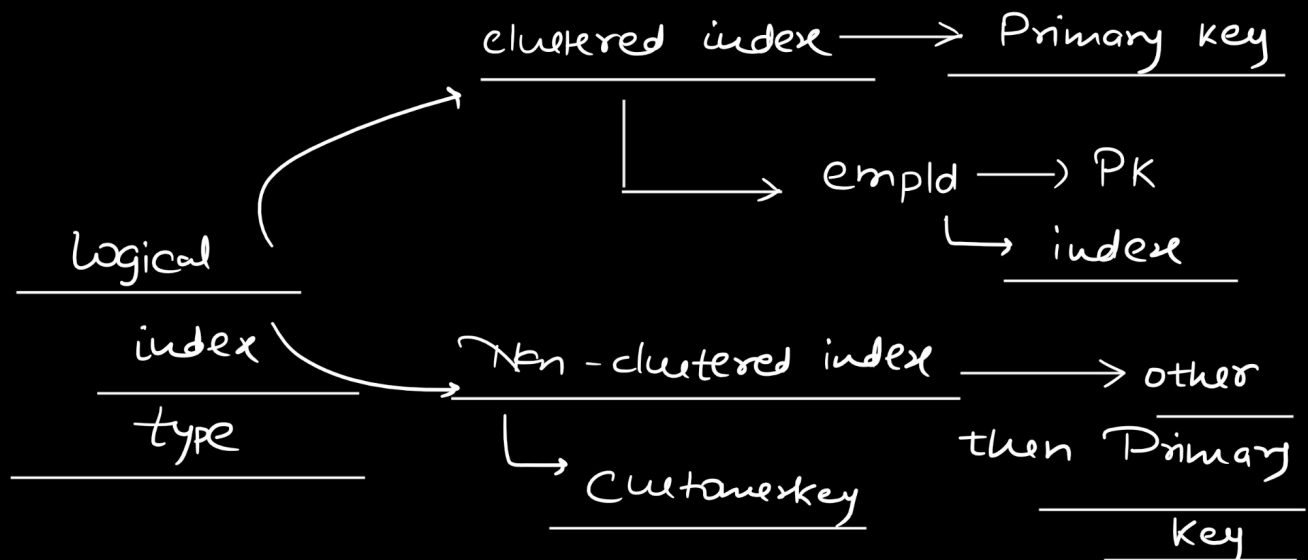
An index is a data structure that allows us to add indexes to the existing table. It enables you to improve the faster retrieval of records on a database table. It creates an entry for each value of the indexed columns. We use it to quickly find the record without searching each row in a database table whenever the table is accessed. We can create an index by using one or more columns of the table for efficient access to the records. An index creates a data structure that improves the speed of data retrieval operations at the cost of additional writes and storage space to maintain the index data structure.

trade off

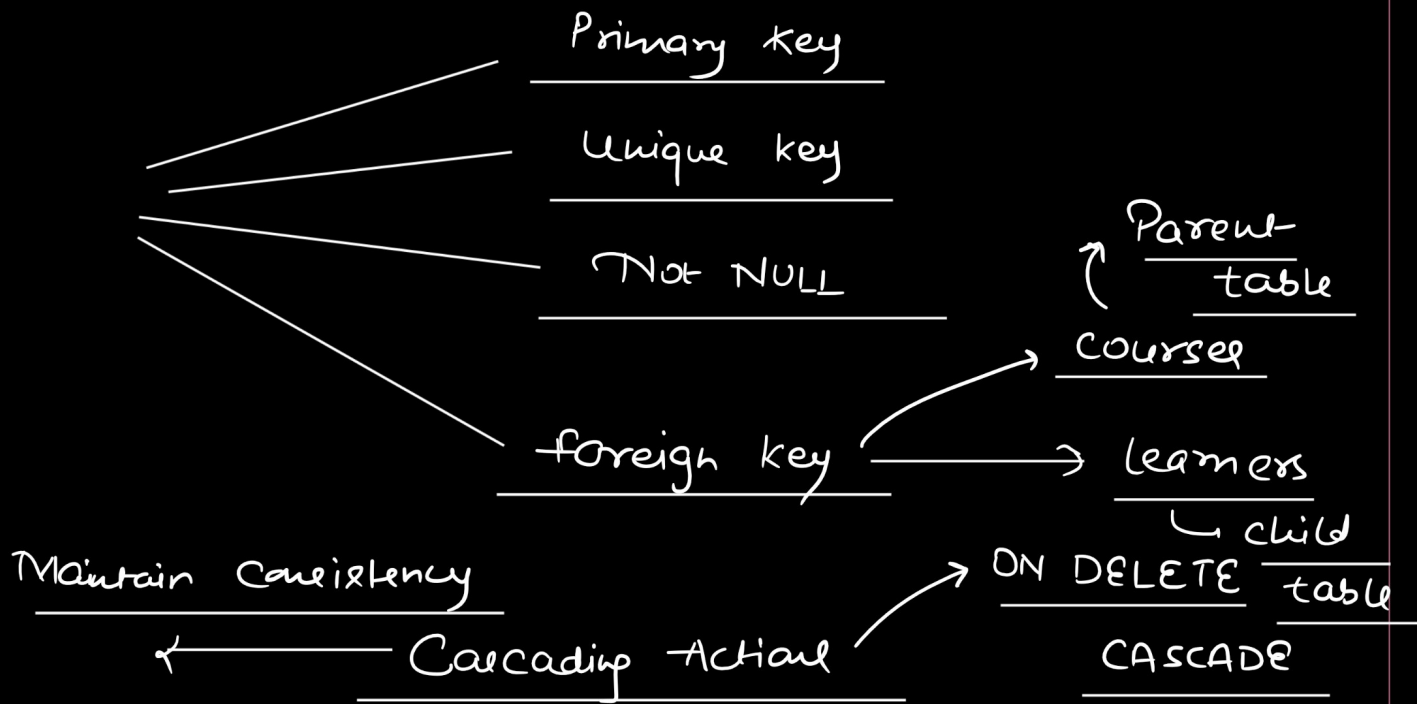
time complexity ↓

space complexity

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
customers	1	CustomerKey	1	CustomerKey	A	2062	HULL	HULL	YES	BTREE			YES	HULL



Constraints in SQL



```
CREATE TABLE Learners(
Learner_Id INT AUTO_INCREMENT,
LearnerFirstName VARCHAR(50) NOT NULL,
LearnerLastName VARCHAR(50) NOT NULL,
LearnerPhoneNo VARCHAR(15) NOT NULL,
LearnerEmailID VARCHAR(50),
LearnerEnrollmentDate TIMESTAMP NOT NULL,
SelectedCourses INT NOT NULL,
YearsOfExperience INT NOT NULL,
LearnerCompany VARCHAR(50),
SourceOfJoining VARCHAR(50) NOT NULL,
Batch_Start_Date TIMESTAMP NOT NULL,
Location VARCHAR(50) NOT NULL,
PRIMARY KEY(Learner_Id),
UNIQUE KEY(LearnerEmailID),
FOREIGN KEY(SelectedCourses) REFERENCES courses(courseID)
ON DELETE CASCADE);
```

When 'ON DELETE CASCADE' is used, deleting a record in the parent table automatically deletes all related records in the child table.

UPDATE CASCADE

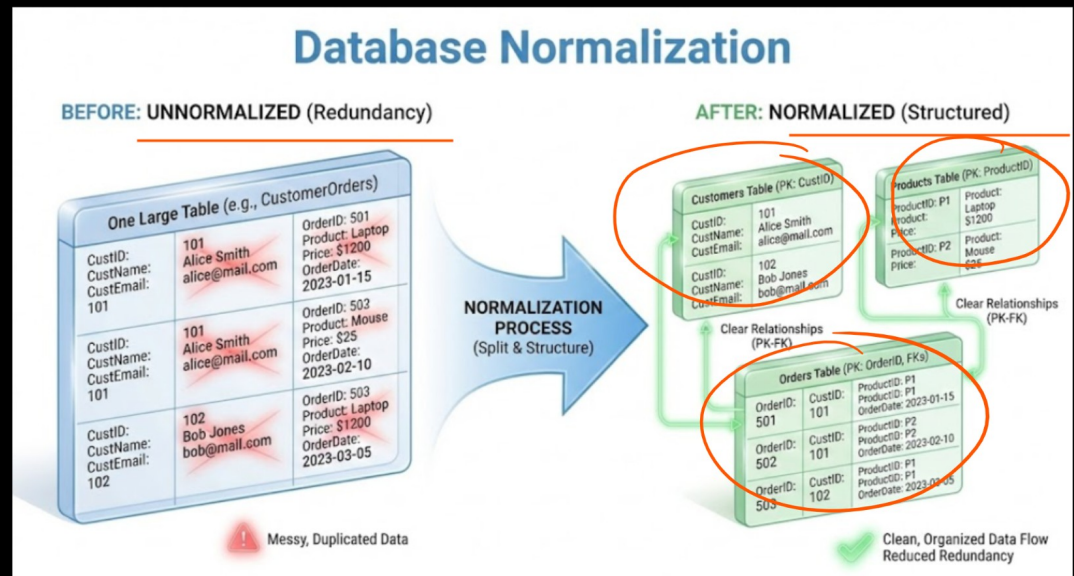
ON DELETE

RESTRICT

This action prevents deletion of a parent record if related child records exist.

```
CREATE TABLE Learners(
Learner_Id INT AUTO_INCREMENT,
LearnerFirstName VARCHAR(50) NOT NULL,
LearnerLastName VARCHAR(50) NOT NULL,
LearnerPhoneNo VARCHAR(15) NOT NULL,
LearnerEmailID VARCHAR(50),
LearnerEnrollmentDate TIMESTAMP NOT NULL,
SelectedCourses INT NOT NULL,
YearsOfExperience INT NOT NULL,
LearnerCompany VARCHAR(50),
SourceOfJoining VARCHAR(50) NOT NULL,
Batch_Start_Date TIMESTAMP NOT NULL,
Location VARCHAR(50) NOT NULL,
PRIMARY KEY(Learner_Id),
UNIQUE KEY(LearnerEmailID),
FOREIGN KEY(SelectedCourses) REFERENCES courses(courseID)
ON DELETE RESTRICT);
```

Normalization



Database normalization is a design approach used to organize data in a structured and logical way. Its primary goal is to ensure that data is stored efficiently, accurately, and without unnecessary duplication.